

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 10%

Mock Interview Questions & Detailed Solutions

13. You need to build a simple face recognition demo. Design a minimal working pipeline, explaining each stage and your design decisions.

Answer:

Pipeline Overview:

Input Frame → Haar/DNN Detection → Align & Crop → Feature Extraction (LBPH / Deep Embeddings) → Cosine/Euclidean Matching → Annotated Display Output.

```
FUNCTION face_recognition_pipeline(frame, database_embeddings):  
    # Stage 1: Face Detection  
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)  
    face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")  
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)  
  
    FOR (x, y, w, h) IN faces:  
        face_roi = gray[y:y+h, x:x+w]  
        face_resized = cv2.resize(face_roi, (100, 100))  
  
        # Stage 2: Feature Extraction & Prediction using LBPH  
        lbph_model = cv2.face.LBPHFaceRecognizer_create()  
        label, confidence = lbph_model.predict(face_resized)  
  
        # Stage 3: Decision Thresholding  
        IF confidence < 50: # Lower distance score implies higher similarity  
            name = database_embeddings.get(label, "Unknown")  
        ELSE:  
            name = "Unknown"  
  
        # Stage 4: Visualization & Bounding Box  
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)  
        cv2.putText(frame, name, (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)  
  
    RETURN frame
```

Detailed Stage Explanations & Design Justifications:

- **Stage 1 (Detection & Preprocessing):** OpenCV's Haar Cascades or DNN face detector locates target faces. Converting to grayscale and resizing to standard dimensions (100x100) minimizes computational overhead and ensures uniform inputs across variable camera sensor inputs.
- **Stage 2 (Feature Extraction):** Local Binary Patterns Histograms (LBPH) extract micro-texture descriptions by comparing local pixels with neighbors. It remains computationally light, highly suitable for real-time edge hardware, and invariant to monotonic grayscale lighting shifts.

- **Stage 3 (Metric Matching & Thresholding):** Chi-square or Euclidean distance compares extracted histogram features with enrolled samples. Setting a distance threshold (< 50) prevents false positive assignments for un-enrolled background subjects.
- **Stage 4 (Visualization):** Overlays bounding regions and associated identity text dynamically onto the BGR frame buffer before pipeline display.

11. You need to compare two images and highlight differences visually. Analyze the problem and design a solution, justifying your approach.

Answer:

Comparing two images to detect visual discrepancies requires handling illumination fluctuations, slight structural alignments, and noise while isolating actual structural changes (e.g., defective assembly lines, document tamper detection, scene modification).

```
FUNCTION highlight_image_differences(img1, img2, threshold_value=25):
    # Ensure equal dimensions
    IF img1.shape != img2.shape:
        img2 = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

    # 1. Convert to Grayscale & Apply Mild Gaussian Blur
    gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
    gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
    gray1_blur = cv2.GaussianBlur(gray1, (5, 5), 0)
    gray2_blur = cv2.GaussianBlur(gray2, (5, 5), 0)

    # 2. Compute Absolute Per-Pixel Difference
    diff = cv2.absdiff(gray1_blur, gray2_blur)

    # 3. Binary Thresholding & Morphological Cleanup
    _, thresh = cv2.threshold(diff, threshold_value, 255, cv2.THRESH_BINARY)
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    cleaned_thresh = cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)

    # 4. Contour Detection and Bounding Box Drawing
    contours, _ = cv2.findContours(cleaned_thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    output_img = img2.copy()

    FOR cnt IN contours:
        IF cv2.contourArea(cnt) > 100: # Filter trivial pixel noise
            x, y, w, h = cv2.boundingRect(cnt)
            cv2.rectangle(output_img, (x, y), (x + w, y + h), (0, 0, 255), 2)

    RETURN output_img, diff
```

Design Justifications:

- **Gaussian Blurring:** Softens sensor noise and minor pixel jitter between shots, reducing false change alerts.
- **Absolute Difference (*cv2.absdiff*):** Produces unsigned intensity delta vectors invariant to direction of brightness changes.
- **Morphological Closing:** Fills small interior voids in detected change patches to unify fragmented sub-regions into single cohesive contours.
- **Area Thresholding:** Rejects minor residual artifacts below 100 pixels, ensuring highlighting focuses solely on meaningful structural variances.

3. You need to overlay text and shapes on a live video feed. Design your solution and explain how you would implement it efficiently, including key OpenCV functions and reasoning.

Answer:

Adding real-time graphical overlays (bounding boxes, ROI polygons, telemetry, frame counters) to high-resolution live video feeds demands low latency to prevent dropping target framerates (e.g., 30–60 FPS).

```
FUNCTION render_live_overlays(video_source=0):
    cap = cv2.VideoCapture(video_source)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    fps_start_time = cv2.getTickCount()
    frame_count = 0
    fps = 0.0

    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        frame_count += 1
        IF frame_count % 10 == 0:
            ticks = (cv2.getTickCount() - fps_start_time) / cv2.getTickFrequency()
            fps = 10.0 / ticks
            fps_start_time = cv2.getTickCount()

        # Draw Bounding Box & Polygon Overlay directly on continuous memory buffer
        cv2.rectangle(frame, (100, 100), (400, 400), (0, 255, 0), 2)
        cv2.circle(frame, (250, 250), 8, (0, 0, 255), -1)

        # Overlay Telemetry Text
        status_text = f"FPS: {fps:.1f} | Stream: Active"
        cv2.putText(frame, status_text, (20, 40), cv2.FONT_HERSHEY_SIMPLEX,
                    0.7, (255, 255, 255), 2, cv2.LINE_AA)

        cv2.imshow("Optimized Overlay Feed", frame)
        IF cv2.waitKey(1) & 0xFF == 27: BREAK

    cap.release()
    cv2.destroyAllWindows()
```

Key OpenCV Functions & Efficiency Reasoning:

- **Direct Buffer Manipulation (*cv2.rectangle*, *cv2.circle*):** In-place pixel modification on contiguous C-memory arrays avoids redundant memory allocation and copy overhead.
- ***cv2.LINE_AA* Anti-aliasing:** Uses Gaussian anti-aliased drawing for crisp rendering without noticeable performance degradation on modern CPUs.
- **Sub-sampled Telemetry Calculations:** Re-computing frame rate metrics every 10 frames reduces timing overhead per iteration while maintaining smooth visual updates.

4. An image appears too dark when read. Analyze the issue and propose modifications to your processing pipeline, justifying how your approach improves visibility.

Answer:

Standard linear contrast adjustments ($I_{out} = \alpha I + \beta$) multiply global pixel intensities, causing bright regions to saturate into severe highlight clipping while noise in dark regions becomes magnified.

Proposed Pipeline Architecture:

Raw Dark Image → Convert to LAB → CLAHE (L-Channel) → Denoise (Non-Local Means) → Gamma Correction → Reconvert to BGR

```
FUNCTION enhance_dark_image(img):
    # 1. Convert to LAB color space to decouple luminance from chromaticity
    lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
    l, a, b = cv2.split(lab)

    # 2. Contrast Limited Adaptive Histogram Equalization (CLAHE)
    # Operates on local sub-tiles to elevate shadow detail without over-exposing highlights
    clahe = cv2.createCLAHE(clipLimit=3.0, tileGridSize=(8, 8))
    l_enhanced = clahe.apply(l)

    # 3. Non-Local Means Denoising on the enhanced luminance channel
    l_denoised = cv2.fastNlMeansDenoising(l_enhanced, None, h=7, templateWindowSize=7,
searchWindowSize=21)

    # 4. Non-linear Gamma Correction ( $\gamma < 1.0$ ) to boost mid-tones
    gamma = 0.6
    inv_gamma = 1.0 / gamma
    table = np.array([((i / 255.0) ** inv_gamma) * 255 FOR i IN RANGE(256)], dtype='uint8')
    l_gamma = cv2.LUT(l_denoised, table)

    # Reconstruct final image
    merged = cv2.merge((l_gamma, a, b))
    RETURN cv2.cvtColor(merged, cv2.COLOR_LAB2BGR)
```

Justification of Improvements:

- **Decoupled Color Space (LAB):** Processing luminance (L) independently prevents color hue shift distortions common in naive RGB gain adjustments.
- **CLAHE Local Equalization:** Divides image into 8x8 grids, clipping local histograms at threshold 3.0 to expand shadow detail while avoiding highlight burnout.
- **LUT-Based Gamma Correction:** Precomputed lookup tables achieve non-linear brightness remapping with low computational complexity.

7. A system must process both images and videos using the same pipeline. Propose a generalized design and explain how it handles both cases effectively.

Answer:

Designing a unified computer vision architecture requires abstracting input sources behind a common interface, treating single static images as single-frame streams while handling temporal video streams naturally within an identical processing pipeline.

```
CLASS UnifiedVisionPipeline:
    DEF __init__(self, processing_func):
        self.processor = processing_func

    DEF process_source(self, source_path):
        # Determine if input is image or video stream
        IF isinstance(source_path, str) AND source_path.lower().endswith(('.jpg', '.png',
'.bmp', '.jpeg')):
            img = cv2.imread(source_path)
            IF img IS NOT NONE:
                processed = self.processor(img)
                cv2.imshow("Unified Output", processed)
                cv2.waitKey(0)
                cv2.destroyAllWindows()
```

```

ELSE:
    # Treat as video file or live stream camera index
    cap = cv2.VideoCapture(source_path)
    WHILE cap.isOpened():
        ret, frame = cap.read()
        IF NOT ret: BREAK

        processed_frame = self.processor(frame)
        cv2.imshow("Unified Output", processed_frame)
        IF cv2.waitKey(1) & 0xFF == 27: BREAK
    cap.release()
    cv2.destroyAllWindows()

# Sample Core Processing Stage
DEF sample_core_stage(frame):
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 100, 200)
    RETURN cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)

```

Design Advantages & Effective Handling:

- **Decoupled Core Algorithmic Logic:** Core computer vision functions operate on generic 3D NumPy array frame buffers without knowledge of source origin.
- **Polymorphic Source Abstraction:** Encapsulates input handling, allowing seamfree switching from offline batch image evaluations to live camera feeds.
- **Resource Management:** Automatically manages window events and video stream handle cleanup.

Code of Conduct:

I DHARSHINI S (192521398) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions	Provides reasonable	Superficial or partially	

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
		critically and provides well-structured, logical answers	analysis with some structure	structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately